

Loop, While, Goto

—Alternative Berechnungsmodelle—

Markus Hecher

Universität Potsdam, Germany

Vorlesung am Apr 20, 2026

Last updated: 2026-04-20

Recap: Turingberechenbarkeit

- verschiedene TM-Modelle, Beschränkungen, Erweiterungen

Recap: Turingberechenbarkeit

- verschiedene TM-Modelle, Beschränkungen, Erweiterungen
- ↪ Gleich “mächtig” (im Sinne der Berechenbarkeit)

Recap: Turingberechenbarkeit

- verschiedene TM-Modelle, Beschränkungen, Erweiterungen
- ~> Gleich “mächtig” (im Sinne der Berechenbarkeit)
- Entscheidbarkeit vs. Semi-Entscheidbarkeit?
- ~> Berechnung charakteristische Funktion vs. partiell charakteristische Funktion

Recap: Turingberechenbarkeit

- verschiedene TM-Modelle, Beschränkungen, Erweiterungen
- ↪ Gleich “mächtig” (im Sinne der Berechenbarkeit)
- Entscheidbarkeit vs. Semi-Entscheidbarkeit?
- ↪ Berechnung charakteristische Funktion vs. partiell charakteristische Funktion
- Deterministisch vs. Nichtdeterministisch?
- ↪ Nichtdeterminismus im Sinne der Berechenbarkeit simulierbar via “Breitensuche” (exponentieller Blowup!)

Recap: Turingberechenbarkeit

- verschiedene TM-Modelle, Beschränkungen, Erweiterungen
- ↪ Gleich “mächtig” (im Sinne der Berechenbarkeit)
- Entscheidbarkeit vs. Semi-Entscheidbarkeit?
- ↪ Berechnung charakteristische Funktion vs. partiell charakteristische Funktion
- Deterministisch vs. Nichtdeterministisch?
- ↪ Nichtdeterminismus im Sinne der Berechenbarkeit simulierbar via “Breitensuche” (exponentieller Blowup!)
- Sprache einer TM M : $L(M) = \{w \in \Sigma^+ \mid q \in F, (\varepsilon, q_0, w) \vdash^* (\varepsilon, q, \beta)\}$

Recap: Turingberechenbarkeit

- verschiedene TM-Modelle, Beschränkungen, Erweiterungen
- ↪ Gleich “mächtig” (im Sinne der Berechenbarkeit)
- Entscheidbarkeit vs. Semi-Entscheidbarkeit?
- ↪ Berechnung charakteristische Funktion vs. partiell charakteristische Funktion
- Deterministisch vs. Nichtdeterministisch?
- ↪ Nichtdeterminismus im Sinne der Berechenbarkeit simulierbar via “Breitensuche” (exponentieller Blowup!)
- Sprache einer TM M :
$$L(M) = \{w \in \Sigma^+ \mid q \in F, (\varepsilon, q_0, w) \vdash^* (\varepsilon, q, \beta)\} \cup \{\varepsilon \mid q \in F, (\varepsilon, q_0, B) \vdash^* (\varepsilon, q, \beta)\}$$

Recap: Turingberechenbarkeit

- verschiedene TM-Modelle, Beschränkungen, Erweiterungen
- ↪ Gleich “mächtig” (im Sinne der Berechenbarkeit)
- Entscheidbarkeit vs. Semi-Entscheidbarkeit?
- ↪ Berechnung charakteristische Funktion vs. partiell charakteristische Funktion
- Deterministisch vs. Nichtdeterministisch?
- ↪ Nichtdeterminismus im Sinne der Berechenbarkeit simulierbar via “Breitensuche” (exponentieller Blowup!)
- Sprache einer TM M :
$$L(M) = \{w \in \Sigma^+ \mid q \in F, (\varepsilon, q_0, w) \vdash^* (\varepsilon, q, \beta)\} \cup \{\varepsilon \mid q \in F, (\varepsilon, q_0, B) \vdash^* (\varepsilon, q, \beta)\}$$
- ↪ Mühsames Schieben von Symbolen ...

Recap: Verdoppelung (unär realisiert, Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, X, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B	X
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)	—
q_1	(q_2, X, R)	(q_4, B, R)	—
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$	—
q_3	$(q_3, 1, L)$	—	$(q_1, 1, L)$
q_4^*	—	—	—

Recap: Verdoppelung (unär realisiert, Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, X, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B	X
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)	—
q_1	(q_2, X, R)	(q_4, B, R)	—
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$	—
q_3	$(q_3, 1, L)$	—	$(q_1, 1, L)$
q_4^*	—	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash$

Recap: Verdoppelung (unär realisiert, Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, X, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B	X
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)	—
q_1	(q_2, X, R)	(q_4, B, R)	—
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$	—
q_3	$(q_3, 1, L)$	—	$(q_1, 1, L)$
q_4^*	—	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash (1X, q_2, B) \vdash (1, q_3, X1) \vdash (\varepsilon, q_1, 111) \vdash$

Recap: Verdoppelung (unär realisiert, Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, X, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B	X
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)	—
q_1	(q_2, X, R)	(q_4, B, R)	—
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$	—
q_3	$(q_3, 1, L)$	—	$(q_1, 1, L)$
q_4^*	—	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash (1X, q_2, B) \vdash (1, q_3, X1) \vdash (\varepsilon, q_1, 111) \vdash (X, q_2, 11) \vdash (X1, q_2, 1) \vdash (X11, q_2, B) \vdash$

Recap: Verdoppelung (unär realisiert, Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, X, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B	X
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)	—
q_1	(q_2, X, R)	(q_4, B, R)	—
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$	—
q_3	$(q_3, 1, L)$	—	$(q_1, 1, L)$
q_4^*	—	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash (1X, q_2, B) \vdash (1, q_3, X1) \vdash (\varepsilon, q_1, 111) \vdash (X, q_2, 11) \vdash (X1, q_2, 1) \vdash (X11, q_2, B) \vdash (X1, q_3, 11) \vdash (X, q_3, 111) \vdash (\varepsilon, q_3, X111) \vdash (\varepsilon, q_1, B1111) \vdash (\varepsilon, q_4, 1111)$

Recap: Verdoppelung (unär realisiert, Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, X, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B	X
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)	—
q_1	(q_2, X, R)	(q_4, B, R)	—
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$	—
q_3	$(q_3, 1, L)$	—	$(q_1, 1, L)$
q_4^*	—	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash (1X, q_2, B) \vdash (1, q_3, X1) \vdash (\varepsilon, q_1, 111) \vdash (X, q_2, 11) \vdash (X1, q_2, 1) \vdash (X11, q_2, B) \vdash (X1, q_3, 11) \vdash (X, q_3, 111) \vdash (\varepsilon, q_3, X111) \vdash (\varepsilon, q_1, B1111) \vdash (\varepsilon, q_4, 1111)$

$\rightsquigarrow f : \mathbb{N} \rightarrow \mathbb{N}$ mit $f(n) = 2n$ Turing-berechenbar

Recap: Verdoppelung (unär realisiert, ohne Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)
q_1	(q_2, B, R)	(q_4, B, R)
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$
q_3	$(q_3, 1, L)$	$(q_1, 1, L)$
q_4^*	—	—

Recap: Verdoppelung (unär realisiert, ohne Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)
q_1	(q_2, B, R)	(q_4, B, R)
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$
q_3	$(q_3, 1, L)$	$(q_1, 1, L)$
q_4^*	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash$

Recap: Verdoppelung (unär realisiert, ohne Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)
q_1	(q_2, B, R)	(q_4, B, R)
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$
q_3	$(q_3, 1, L)$	$(q_1, 1, L)$
q_4^*	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash (1B, q_2, B) \vdash (1, q_3, B1) \vdash (\varepsilon, q_1, 111) \vdash$

Recap: Verdoppelung (unär realisiert, ohne Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)
q_1	(q_2, B, R)	(q_4, B, R)
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$
q_3	$(q_3, 1, L)$	$(q_1, 1, L)$
q_4^*	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash (1B, q_2, B) \vdash (1, q_3, B1) \vdash (\varepsilon, q_1, 111) \vdash (\varepsilon, q_2, 11) \vdash (1, q_2, 1) \vdash (11, q_2, B) \vdash$

Recap: Verdoppelung (unär realisiert, ohne Marker X)

$M_{n \rightarrow 2n} = (\{q_0, q_1, q_2, q_3, q_4\}, \{1\}, \{1, B\}, \delta_4, q_0, B, \{q_4\})$ mit

δ_4	1	B
$\rightarrow q_0$	$(q_0, 1, R)$	(q_1, B, L)
q_1	(q_2, B, R)	(q_4, B, R)
q_2	$(q_2, 1, R)$	$(q_3, 1, L)$
q_3	$(q_3, 1, L)$	$(q_1, 1, L)$
q_4^*	—	—

- Abarbeitungsbeispiel: $(\varepsilon, q_0, 11) \vdash (1, q_0, 1) \vdash (11, q_0, B) \vdash (1, q_1, 1) \vdash (1B, q_2, B) \vdash (1, q_3, B1) \vdash (\varepsilon, q_1, 111) \vdash (\varepsilon, q_2, 11) \vdash (1, q_2, 1) \vdash (11, q_2, B) \vdash (1, q_3, 11) \vdash (\varepsilon, q_3, 111) \vdash (\varepsilon, q_3, B111) \vdash (\varepsilon, q_1, B1111) \vdash (\varepsilon, q_4, 1111)$

Motivation

Heute?

- Alternative Berechenbarkeitsmodelle
- Orientierung an imperativen Programmiersprachen, warum?

Heute?

- Alternative Berechenbarkeitsmodelle
- Orientierung an imperativen Programmiersprachen, warum?
- ↪ TMs wie Hardware, primitive Modelle, daher “Programme” (δ) überladen mit viel zu vielen Details

Heute?

- Alternative Berechenbarkeitsmodelle
- Orientierung an imperativen Programmiersprachen, warum?
- ↪ TMs wie Hardware, primitive Modelle, daher “Programme” (δ) überladen mit viel zu vielen Details

Church-Turing These: Die Menge der Turing-berechenbaren Funktionen stimmt mit der Menge der intuitiv berechenbaren Funktionen überein.

Heute?

- Alternative Berechenbarkeitsmodelle
- Orientierung an imperativen Programmiersprachen, warum?
- ↪ TMs wie Hardware, primitive Modelle, daher “Programme” (δ) überladen mit viel zu vielen Details

Church-Turing These: Die Menge der Turing-berechenbaren Funktionen stimmt mit der Menge der intuitiv berechenbaren Funktionen überein.

- Versuch, genau die total berechenbaren Funktionen zu charakterisieren

Heute?

- Alternative Berechenbarkeitsmodelle
- Orientierung an imperativen Programmiersprachen, warum?
- ↪ TMs wie Hardware, primitive Modelle, daher “Programme” (δ) überladen mit viel zu vielen Details

Church-Turing These: Die Menge der Turing-berechenbaren Funktionen stimmt mit der Menge der intuitiv berechenbaren Funktionen überein.

- Versuch, genau die total berechenbaren Funktionen zu charakterisieren
- ↪ Wie mächtig sind diese im Vergleich untereinander und mit den TMs?

Totale Berechenbarkeit

- ↪ Gleiches Berechenbarkeitsmodell (TMs) für verschiedene Berechenbarkeitsbegriffe (totale Turing-Berechenbarkeit und Turing-Berechenbarkeit)

Totale Berechenbarkeit

- ↪ Gleiches Berechenbarkeitsmodell (TMs) für verschiedene Berechenbarkeitsbegriffe (totale Turing-Berechenbarkeit und Turing-Berechenbarkeit)
 - Es gibt Funktionen, die durch TMs berechnet werden, die stets anhalten

Totale Berechenbarkeit

- ~> Gleiches Berechenbarkeitsmodell (TMs) für verschiedene Berechenbarkeitsbegriffe (totale Turing-Berechenbarkeit und Turing-Berechenbarkeit)
 - Es gibt Funktionen, die durch TMs berechnet werden, die stets anhalten
 - A priori nicht feststellbar, für welche TMs das zutrifft (kein äußeres Merkmal)
- ~> Unentscheidbares Halteproblem: Wird eine TM schlussendlich irgendwann halten?

Totale Berechenbarkeit

- ↪ Gleiches Berechenbarkeitsmodell (TMs) für verschiedene Berechenbarkeitsbegriffe (totale Turing-Berechenbarkeit und Turing-Berechenbarkeit)
 - Es gibt Funktionen, die durch TMs berechnet werden, die stets anhalten
 - A priori nicht feststellbar, für welche TMs das zutrifft (kein äußeres Merkmal)
- ↪ Unentscheidbares Halteproblem: Wird eine TM schlussendlich irgendwann halten?

- ↪ Versuch, der Form nach unterscheidbare Berechenbarkeitsmodelle zu finden und zu vergleichen

Loop-Programme

Teaser: Addition

Programm $\text{add}(x_1, x_2)$:

Teaser: Addition

Programm $\text{add}(x_1, x_2)$:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

Teaser: Addition

Programm $\text{add}(x_1, x_2)$:

$x_0 := x_1$; // Zuweisungsmakro, Definition später

loop x_2 **do**

Teaser: Addition

Programm $\text{add}(x_1, x_2)$:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := \text{suc}(x_0)$

end

↪ Im Vergleich zur TM: kompakt!

Loop-Programme

Das Berechenbarkeitsmodell

Syntaktische Elemente

- Loop-Programme bestehen aus folgenden Komponenten:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$

Syntaktische Elemente

- Loop-Programme bestehen aus folgenden Komponenten:
 - der Konstanten **0**
 - den Variablen **x_i** für $i \in \mathbb{N}$
 - den einstelligen Operatoren ***suc*** und ***pred***

Syntaktische Elemente

- Loop-Programme bestehen aus folgenden Komponenten:
 - der Konstanten **0**
 - den Variablen **x_i** für $i \in \mathbb{N}$
 - den einstelligen Operatoren ***suc*** und ***pred***
 - dem Zuweisungsoperator **$:=$**

Syntaktische Elemente

- Loop-Programme bestehen aus folgenden Komponenten:
 - der Konstanten **0**
 - den Variablen **x_i** für $i \in \mathbb{N}$
 - den einstelligen Operatoren ***suc*** und ***pred***
 - dem Zuweisungsoperator **$:=$**
 - dem Kompositionsoperator **$;$**
 - dem Schleifenkonstrukt **loop do end**

Syntaktische Elemente

- Loop-Programme bestehen aus folgenden Komponenten:
 - der Konstanten **0**
 - den Variablen **x_i** für $i \in \mathbb{N}$
 - den einstelligen Operatoren ***suc*** und ***pred***
 - dem Zuweisungsoperator **$:=$**
 - dem Kompositionsoperator **$;$**
 - dem Schleifenkonstrukt **loop do end**

⇒ $\mathcal{LP}$ = Menge aller Loop-Programme

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

- $N = \{ \langle \text{Loop-Programm} \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Zuweisung} \rangle, \langle \text{Variable} \rangle \}$

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

- $N = \{ \langle \text{Loop-Programm} \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Zuweisung} \rangle, \langle \text{Variable} \rangle \}$
- T ist das deutsche Alphabet mit Leerstelle, Klammern, $' := '$ und $' ; '$

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

- $N = \{ \langle \text{Loop-Programm} \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Zuweisung} \rangle, \langle \text{Variable} \rangle \}$
- T ist das deutsche Alphabet mit Leerstelle, Klammern, $' := '$ und $' ; '$
- $P = \{ \langle \text{Loop-Programm} \rangle \rightarrow \varepsilon \mid \langle \text{Anweisungsblock} \rangle ; \langle \text{Loop-Programm} \rangle,$

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

- $N = \{ \langle \text{Loop-Programm} \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Zuweisung} \rangle, \langle \text{Variable} \rangle \}$
- T ist das deutsche Alphabet mit Leerstelle, Klammern, $' := '$ und $' ; '$
- $P = \{$
 $\langle \text{Loop-Programm} \rangle \rightarrow \varepsilon \mid \langle \text{Anweisungsblock} \rangle ; \langle \text{Loop-Programm} \rangle,$
 $\langle \text{Anweisungsblock} \rangle \rightarrow \langle \text{Zuweisung} \rangle \mid \mathbf{loop} \langle \text{Variable} \rangle \mathbf{do} \langle \text{Loop-Programm} \rangle \mathbf{end},$

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

- $N = \{ \langle \text{Loop-Programm} \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Zuweisung} \rangle, \langle \text{Variable} \rangle \}$
- T ist das deutsche Alphabet mit Leerstelle, Klammern, ':= ' und ';
- $P = \{$
 $\langle \text{Loop-Programm} \rangle \rightarrow \varepsilon \mid \langle \text{Anweisungsblock} \rangle ; \langle \text{Loop-Programm} \rangle,$
 $\langle \text{Anweisungsblock} \rangle \rightarrow \langle \text{Zuweisung} \rangle \mid \mathbf{loop} \langle \text{Variable} \rangle \mathbf{do} \langle \text{Loop-Programm} \rangle \mathbf{end},$
 $\langle \text{Zuweisung} \rangle \rightarrow \langle \text{Variable} \rangle := 0 \mid \langle \text{Variable} \rangle := \text{suc}(\langle \text{Variable} \rangle) \mid$
 $\langle \text{Variable} \rangle := \text{pred}(\langle \text{Variable} \rangle),$

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

- $N = \{ \langle \text{Loop-Programm} \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Zuweisung} \rangle, \langle \text{Variable} \rangle \}$
- T ist das deutsche Alphabet mit Leerstelle, Klammern, ':= ' und ';
- $P = \{$
 $\langle \text{Loop-Programm} \rangle \rightarrow \varepsilon \mid \langle \text{Anweisungsblock} \rangle ; \langle \text{Loop-Programm} \rangle,$
 $\langle \text{Anweisungsblock} \rangle \rightarrow \langle \text{Zuweisung} \rangle \mid \mathbf{loop} \langle \text{Variable} \rangle \mathbf{do} \langle \text{Loop-Programm} \rangle \mathbf{end},$
 $\langle \text{Zuweisung} \rangle \rightarrow \langle \text{Variable} \rangle := 0 \mid \langle \text{Variable} \rangle := \text{suc}(\langle \text{Variable} \rangle) \mid$
 $\langle \text{Variable} \rangle := \text{pred}(\langle \text{Variable} \rangle),$
 $\langle \text{Variable} \rangle \rightarrow x_0 \mid \dots \mid x_n \}$

Formalere Syntax-Beschreibung ...

... via CFG $(N, T, P, \langle \text{Loop-Programm} \rangle)$ möglich

- $N = \{ \langle \text{Loop-Programm} \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Zuweisung} \rangle, \langle \text{Variable} \rangle \}$
- T ist das deutsche Alphabet mit Leerstelle, Klammern, ':= ' und ';
- $P = \{$
 $\langle \text{Loop-Programm} \rangle \rightarrow \varepsilon \mid \langle \text{Anweisungsblock} \rangle ; \langle \text{Loop-Programm} \rangle,$
 $\langle \text{Anweisungsblock} \rangle \rightarrow \langle \text{Zuweisung} \rangle \mid \textbf{loop} \langle \text{Variable} \rangle \textbf{do} \langle \text{Loop-Programm} \rangle \textbf{end},$
 $\langle \text{Zuweisung} \rangle \rightarrow \langle \text{Variable} \rangle := 0 \mid \langle \text{Variable} \rangle := \text{suc}(\langle \text{Variable} \rangle) \mid$
 $\langle \text{Variable} \rangle := \text{pred}(\langle \text{Variable} \rangle),$
 $\langle \text{Variable} \rangle \rightarrow x_0 \mid \dots \mid x_n \}$
- Letztes Semikolon eines Programms wird oft weggelassen

Semantik I

- Speichervektordarstellung $v = (\underbrace{x_0}_{\text{Ergebnis}}, \underbrace{x_1, \dots, x_n}_{\text{Eingaben}}, \underbrace{x_{n+1}, x_{n+2}, \dots}_{\text{Hilfsvariablen}})$

Semantik I

■ Speichervektordarstellung $v = (\underbrace{x_0}_{\text{Ergebnis}}, \underbrace{x_1, \dots, x_n}_{\text{Eingaben}}, \underbrace{x_{n+1}, x_{n+2}, \dots}_{\text{Hilfsvariablen}})$

$\rightsquigarrow v : \mathbb{N} \rightarrow \mathbb{N}$

Semantik I

■ Speichervektordarstellung $v = (\underbrace{x_0}_{\text{Ergebnis}}, \underbrace{x_1, \dots, x_n}_{\text{Eingaben}}, \underbrace{x_{n+1}, x_{n+2}, \dots}_{\text{Hilfsvariablen}})$

↪ $v : \mathbb{N} \rightarrow \mathbb{N}$

↪ Semantik via $\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$

Semantik I

- Speichervektordarstellung $v = (\underbrace{x_0}_{\text{Ergebnis}}, \underbrace{x_1, \dots, x_n}_{\text{Eingaben}}, \underbrace{x_{n+1}, x_{n+2}, \dots}_{\text{Hilfsvariablen}})$

↪ $v : \mathbb{N} \rightarrow \mathbb{N}$

↪ Semantik via $\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$

- Hilfsnotation $v[y/x_i]$: Vektor v , bei dem y die Stelle x_i ersetzt

Semantik II

$$\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

- Leeres Programm: $\delta(v, \varepsilon) := v$

Semantik II

$$\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

- Leeres Programm: $\delta(v, \varepsilon) := v$
- Zuweisungen:
 - $\delta(v, x_i := 0) := v[0/x_i]$

Semantik II

$$\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

- Leeres Programm: $\delta(v, \varepsilon) := v$
- Zuweisungen:
 - $\delta(v, x_i := 0) := v[0/x_i]$
 - $\delta(v, x_i := \text{succ}(x_j)) := v[x_j + 1/x_i]$

Semantik II

$$\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

- Leeres Programm: $\delta(v, \varepsilon) := v$
- Zuweisungen:
 - $\delta(v, x_i := 0) := v[0/x_i]$
 - $\delta(v, x_i := \text{succ}(x_j)) := v[x_j + 1/x_i]$
 - $\delta(v, x_i := \text{pred}(x_j)) := \begin{cases} v[x_j - 1/x_i] & \text{falls } x_j \geq 1, \\ v[0/x_i] & \text{sonst} \end{cases}$

Semantik II

$$\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

- Leeres Programm: $\delta(v, \varepsilon) := v$
- Zuweisungen:
 - $\delta(v, x_i := 0) := v[0/x_i]$
 - $\delta(v, x_i := \text{succ}(x_j)) := v[x_j + 1/x_i]$
 - $\delta(v, x_i := \text{pred}(x_j)) := \begin{cases} v[x_j - 1/x_i] & \text{falls } x_j \geq 1, \\ v[0/x_i] & \text{sonst} \end{cases}$
- Komposition: $\delta(v, P_1; P_2) := \delta(\delta(v, P_1), P_2)$,
(P_1 muss ein Anweisungsblock sein, d.h. eine Loop-Schleife oder eine Zuweisung)

Semantik II

$$\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

- Leeres Programm: $\delta(v, \varepsilon) := v$
- Zuweisungen:
 - $\delta(v, x_i := 0) := v[0/x_i]$
 - $\delta(v, x_i := \text{succ}(x_j)) := v[x_j + 1/x_i]$
 - $\delta(v, x_i := \text{pred}(x_j)) := \begin{cases} v[x_j - 1/x_i] & \text{falls } x_j \geq 1, \\ v[0/x_i] & \text{sonst} \end{cases}$
- Komposition: $\delta(v, P_1; P_2) := \delta(\delta(v, P_1), P_2)$,
(P_1 muss ein Anweisungsblock sein, d.h. eine Loop-Schleife oder eine Zuweisung)
- Loop-Schleife: $\delta(v, \text{loop } x_i \text{ do } P \text{ end}) := \delta(v, \underbrace{P; P; \dots; P}_{x_i \text{ Kopien}})$

↪ Dynamische, aber fixierte Anzahl an Schleifeniterationen

Loop-Berechenbarkeit

- Für $n \in \mathbb{N}$ ist $f : \mathbb{N}^n \rightarrow \mathbb{N}$ Loop-berechenbar $\iff$ es gibt $P \in \mathcal{LP}$,

Loop-Berechenbarkeit

- Für $n \in \mathbb{N}$ ist $f : \mathbb{N}^n \rightarrow \mathbb{N}$ Loop-berechenbar $\iff$ es gibt $P \in \mathcal{LP}$,
 $\delta((0, x_1, \dots, x_n, 0, \dots), P)(0) = f(x_1, \dots, x_n)$,

Loop-Berechenbarkeit

- Für $n \in \mathbb{N}$ ist $f : \mathbb{N}^n \rightarrow \mathbb{N}$ Loop-berechenbar $\iff$ es gibt $P \in \mathcal{LP}$,
 $\delta((0, x_1, \dots, x_n, 0, \dots), P)(0) = f(x_1, \dots, x_n)$, für **alle** $x_1, \dots, x_n \in \mathbb{N}$

Loop-Berechenbarkeit

- Für $n \in \mathbb{N}$ ist $f : \mathbb{N}^n \rightarrow \mathbb{N}$ Loop-berechenbar $\iff$ es gibt $P \in \mathcal{LP}$,
 $\delta((0, x_1, \dots, x_n, 0, \dots), P)(0) = f(x_1, \dots, x_n)$, für **alle** $x_1, \dots, x_n \in \mathbb{N}$
- $\rightsquigarrow \mathcal{LF} =$ Menge aller Loop-berechenbaren Funktionen

Loop-Programme

Beispiele

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := \text{suc}(x_0)$

end

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := \text{suc}(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := suc(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

$\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := suc(x_0) \text{ end}) =$

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := suc(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

$\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := suc(x_0) \text{ end}) =$

$\delta(\delta((0, 5, 2, 0, \dots), x_0 := x_1), \text{loop } x_2 \text{ do } x_0 := suc(x_0) \text{ end}) =$

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := suc(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

$\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := suc(x_0) \text{ end}) =$

$\delta(\delta((0, 5, 2, 0, \dots), x_0 := x_1), \text{loop } x_2 \text{ do } x_0 := suc(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), \text{loop } x_2 \text{ do } x_0 := suc(x_0) \text{ end}) =$

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := \text{suc}(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

$\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta(\delta((0, 5, 2, 0, \dots), x_0 := x_1), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), x_0 := \text{suc}(x_0); x_0 := \text{suc}(x_0)) =$

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := \text{suc}(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

$\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta(\delta((0, 5, 2, 0, \dots), x_0 := x_1), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), x_0 := \text{suc}(x_0); x_0 := \text{suc}(x_0)) =$

$\delta(\delta((5, 5, 2, 0, \dots), x_0 := \text{suc}(x_0)), x_0 := \text{suc}(x_0)) =$

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := \text{suc}(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

$\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta(\delta((0, 5, 2, 0, \dots), x_0 := x_1), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), x_0 := \text{suc}(x_0); x_0 := \text{suc}(x_0)) =$

$\delta(\delta((5, 5, 2, 0, \dots), x_0 := \text{suc}(x_0)), x_0 := \text{suc}(x_0)) =$

$\delta((6, 5, 2, 0, \dots), x_0 := \text{suc}(x_0)) =$

Addition I

- Eingaben x_1, x_2

- Programm add:

$x_0 := x_1;$ // Zuweisungsmakro, Definition später

loop x_2 **do**

$x_0 := \text{suc}(x_0)$

end

- $\delta((0, 5, 2, 0, \dots), \text{add}) =$

$\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta(\delta((0, 5, 2, 0, \dots), x_0 := x_1), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), \text{loop } x_2 \text{ do } x_0 := \text{suc}(x_0) \text{ end}) =$

$\delta((5, 5, 2, 0, \dots), x_0 := \text{suc}(x_0); x_0 := \text{suc}(x_0)) =$

$\delta(\delta((5, 5, 2, 0, \dots), x_0 := \text{suc}(x_0)), x_0 := \text{suc}(x_0)) =$

$\delta((6, 5, 2, 0, \dots), x_0 := \text{suc}(x_0)) =$

$(7, 5, 2, 0, \dots)$

Addition II

- Für alle $n, m \in \mathbb{N}$ gilt: $\delta((0, n, m, 0, \dots), \text{add}) = (n + m, n, m, 0, \dots)$, daher

Addition II

- Für alle $n, m \in \mathbb{N}$ gilt: $\delta((0, n, m, 0, \dots), \text{add}) = (n + m, n, m, 0, \dots)$, daher
 $\delta((0, n, m, 0, \dots), \text{add})(0) = n + m$

Addition II

- Für alle $n, m \in \mathbb{N}$ gilt: $\delta((0, n, m, 0, \dots), \text{add}) = (n + m, n, m, 0, \dots)$, daher
 $\delta((0, n, m, 0, \dots), \text{add})(0) = n + m$

$\rightsquigarrow \text{add} \in \mathcal{LF}$

Die Konstantenfunktionen

■ Programm const_n : $\underbrace{x_0 := \text{suc}(x_0); \dots; x_0 := \text{suc}(x_0)}_{n\text{-mal}}$

Die Konstantenfunktionen

- Programm const_n :
$$\underbrace{x_0 := \text{suc}(x_0); \dots; x_0 := \text{suc}(x_0)}_{n\text{-mal}}$$
- Programm unabhängig von Eingaben und deren Anzahl

Die Konstantenfunktionen

■ Programm const_n : $\underbrace{x_0 := \text{suc}(x_0); \dots; x_0 := \text{suc}(x_0)}_{n\text{-mal}}$

■ Programm unabhängig von Eingaben und deren Anzahl

↪ $\text{const}_n^m \in \mathcal{LF}$, wobei $\text{const}_n^m : \mathbb{N}^m \rightarrow \mathbb{N}$ mit $\text{const}_n^m(x_1, \dots, x_m) = n$

Projektionen

- Programm pr_n :
 $x_0 := \text{suc}(x_n)$
 $x_0 := \text{pred}(x_0)$

Projektionen

- Programm pr_n :

$x_0 := \text{suc}(x_n)$

$x_0 := \text{pred}(x_0)$

↪ $\text{pr}_n^m \in \mathcal{LF}$, wobei $\text{pr}_n^m : \mathbb{N}^m \rightarrow \mathbb{N}$ mit $\text{pr}_n^m(x_1, \dots, x_m) = x_n$

Loop-Programme Makros

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index
- Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index
- ↪ Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen
- Ausdruck $x_i := x_j$ // wird zu:
 - $x_i := \text{suc}(x_j);$
 - $x_i := \text{pred}(x_i)$

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index
- Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen
- Ausdruck $x_i := x_j$ // wird zu:
$$x_i := \text{suc}(x_j);$$
$$x_i := \text{pred}(x_i)$$
- Für $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}} \in \mathcal{LP}$,

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index
- Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen
- Ausdruck $x_i := x_j$ // wird zu:
$$x_i := \text{suc}(x_j);$$
$$x_i := \text{pred}(x_i)$$
- Für $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}} \in \mathcal{LP}$, Ausdruck $x_i := \text{op}(x_{j_1}, \dots, x_{j_n})$ // wird zu:

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index
- Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen
- Ausdruck $x_i := x_j$ // wird zu:

$$\begin{aligned}
 &x_i := \text{suc}(x_j); \\
 &x_i := \text{pred}(x_i)
 \end{aligned}$$
- Für $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}} \in \mathcal{LP}$, Ausdruck $x_i := \text{op}(x_{j_1}, \dots, x_{j_n})$ // wird zu:

$$\begin{aligned}
 &x_{I_P+2} := x_{j_1}; \quad \dots \quad x_{I_P+n+1} := x_{j_n}; \quad // \text{ Aufruf vorbereiten}
 \end{aligned}$$

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index
- Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen
- Ausdruck $x_i := x_j$ // wird zu:

$$x_i := \text{suc}(x_j);$$

$$x_i := \text{pred}(x_i)$$
- Für $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}} \in \mathcal{LP}$, Ausdruck $x_i := \text{op}(x_{j_1}, \dots, x_{j_n})$ // wird zu:

$$x_{I_P+2} := x_{j_1}; \quad \dots \quad x_{I_P+n+1} := x_{j_n}; \quad // \text{ Aufruf vorbeiten}$$

$$P'_{\text{op}}; \quad // \text{ Aufruf } P'_{\text{op}}: \text{ gegenüber } P_{\text{op}} \text{ sind alle Indizes um } I_P + 1 \text{ erhöht}$$

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index
- ⇒ Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen
- Ausdruck $x_i := x_j$ // wird zu:
 - $x_i := \text{suc}(x_j);$
 - $x_i := \text{pred}(x_i)$
- Für $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}} \in \mathcal{LP}$, Ausdruck $x_i := \text{op}(x_{j_1}, \dots, x_{j_n})$ // wird zu:
 - $x_{I_P+2} := x_{j_1}; \quad \dots \quad x_{I_P+n+1} := x_{j_n};$ // Aufruf vorbereiten
 - $P'_{\text{op}};$ // Aufruf P'_{op} : gegenüber P_{op} sind alle Indizes um $I_P + 1$ erhöht
 - $x_i := x_{I_P+1};$ // Ergebnis abholen

Makros I

- Sei P das aktuelle Programm mit Makros, I_P der größte in P vorkommende Index

⇒ Wir konstruieren neues Programm P' , wo wir ein Makrovorkommen ersetzen

- Ausdruck $x_i := x_j$ // wird zu:

$x_i := \text{suc}(x_j);$

$x_i := \text{pred}(x_i)$

- Für $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}} \in \mathcal{LP}$, Ausdruck $x_i := \text{op}(x_{j_1}, \dots, x_{j_n})$ // wird zu:

$x_{I_P+2} := x_{j_1}; \quad \dots \quad x_{I_P+n+1} := x_{j_n}; \quad // \text{ Aufruf vorbereiten}$

$P'_{\text{op}}; \quad // \text{ Aufruf } P'_{\text{op}}: \text{ gegenüber } P_{\text{op}} \text{ sind alle Indizes um } I_P + 1 \text{ erhöht}$

$x_i := x_{I_P+1}; \quad // \text{ Ergebnis abholen}$

$x_{I_P+1} := 0; \quad \dots \quad x_{\max(I_P+n+1, I_{P'_{\text{op}}})} := 0 \quad // \text{ Sauber machen!}$

⇒ Vergleichbar mit Callstack-Allokation (Compilerbau)!

Makros I

Beispiel: $x_{76} := (x_{42} + x_{12})$, eingebettet in P mit $I_P = 76$

Makros I

Beispiel: $x_{76} := (x_{42} + x_{12})$, eingebettet in P mit $I_P = 76$

$x_{78} := x_{42}; \quad x_{79} := x_{12}; \quad // \text{ Aufruf vorbereiten}$

Makros I

Beispiel: $x_{76} := (x_{42} + x_{12})$, eingebettet in P mit $I_P = 76$

```
x78 := x42;      x79 := x12;      // Aufruf vorbereiten  
P'add;           // Aufruf, Indices um 77 erhöht
```

Makros I

Beispiel: $x_{76} := (x_{42} + x_{12})$, eingebettet in P mit $I_P = 76$

```
x78 := x42;      x79 := x12;      // Aufruf vorbereiten  
P'add;           // Aufruf, Indices um 77 erhöht  
x76 := x77;      // Ergebnis abholen
```

Makros I

Beispiel: $x_{76} := (x_{42} + x_{12})$, eingebettet in P mit $I_P = 76$

```
x78 := x42;      x79 := x12;      // Aufruf vorbereiten
P'_{add};          // Aufruf, Indices um 77 erhöht
x76 := x77;       // Ergebnis abholen
x77 := 0;         x78 := 0;         x79 := 0      // Sauber machen (Evtl. Hilfsvariablen!)
```

Makros II

↪ Von “Außen” nach “Innen”:

Makros II

→ Von “Außen” nach “Innen”:

- Ausdruck $x_i := \text{op}(\text{op}_1(x_{j_1,1}, \dots, x_{j_1,m_1}), \dots, \text{op}_n(x_{j_n,1}, \dots, x_{j_n,m_n}))$,

Makros II

↪ Von “Außen” nach “Innen”:

- Ausdruck $x_i := \text{op}(\text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}}), \dots, \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}}))$, für $\text{op}_i : \mathbb{N}^{m_i} \rightarrow \mathbb{N}$
und $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}_i}, P_{\text{op}} \in \mathcal{LP}$ // wird zu:

Makros II

↪ Von “Außen” nach “Innen”:

- Ausdruck $x_i := \text{op}(\text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}}), \dots, \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}}))$, für $\text{op}_i : \mathbb{N}^{m_i} \rightarrow \mathbb{N}$
und $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}_i}, P_{\text{op}} \in \mathcal{LP}$ // wird zu:
 $x_{I_P+2} := \text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}});$ // Aufruf vorbereiten, Makros!

Makros II

↪ Von “Außen” nach “Innen”:

- Ausdruck $x_i := \text{op}(\text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}}), \dots, \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}}))$, für $\text{op}_i : \mathbb{N}^{m_i} \rightarrow \mathbb{N}$
und $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}_i}, P_{\text{op}} \in \mathcal{LP}$ // wird zu:

$$x_{I_P+2} := \text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}}); \quad // \text{ Aufruf vorbereiten, Makros!}$$

$$\dots$$

$$x_{I_P+n+1} := \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}});$$

Makros II

↪ Von “Außen” nach “Innen”:

- Ausdruck $x_i := \text{op}(\text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}}), \dots, \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}}))$, für $\text{op}_i : \mathbb{N}^{m_i} \rightarrow \mathbb{N}$
und $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}_i}, P_{\text{op}} \in \mathcal{LP}$ // wird zu:

$x_{I_P+2} := \text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}});$ // Aufruf vorbereiten, Makros!

...

$x_{I_P+n+1} := \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}});$
 $P'_{\text{op}};$ // Aufruf

Makros II

↪ Von “Außen” nach “Innen”:

- Ausdruck $x_i := \text{op}(\text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}}), \dots, \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}}))$, für $\text{op}_i : \mathbb{N}^{m_i} \rightarrow \mathbb{N}$
und $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}_i}, P_{\text{op}} \in \mathcal{LP}$ // wird zu:

$x_{I_P+2} := \text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}});$ // Aufruf vorbereiten, Makros!

...

$x_{I_P+n+1} := \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}});$

$P'_{\text{op}};$ // Aufruf

$x_i := x_{I_P+1};$ // Ergebnis abholen

Makros II

↪ Von “Außen” nach “Innen”:

- Ausdruck $x_i := \text{op}(\text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}}), \dots, \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}}))$, für $\text{op}_i : \mathbb{N}^{m_i} \rightarrow \mathbb{N}$
und $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $P_{\text{op}_i}, P_{\text{op}} \in \mathcal{LP}$ // wird zu:

$x_{l_P+2} := \text{op}_1(x_{j_{1,1}}, \dots, x_{j_{1,m_1}});$ // Aufruf vorbereiten, Makros!

...

$x_{l_P+n+1} := \text{op}_n(x_{j_{n,1}}, \dots, x_{j_{n,m_n}});$

$P'_{\text{op}};$ // Aufruf

$x_i := x_{l_P+1};$ // Ergebnis abholen

$x_{l_P+1} = 0; \quad \dots \quad x_{\max(l_P+n+1, l_{P'_{\text{op}}})} := 0$ // Sauber machen!

Makros II

Beispiel: $x_{73} := (x_{42} + x_{12}) \cdot \text{suc}(x_5)$, eingebettet in P mit $I_P = 75$

```
// Aufruf vorbereiten (Makros!)
```

Makros II

Beispiel: $x_{73} := (x_{42} + x_{12}) \cdot \text{suc}(x_5)$, eingebettet in P mit $I_P = 75$

$x_{77} := x_{42} + x_{12};$

// Aufruf vorbereiten (Makros!)

$x_{78} := \text{suc}(x_5);$

Makros II

Beispiel: $x_{73} := (x_{42} + x_{12}) \cdot \text{suc}(x_5)$, eingebettet in P mit $I_P = 75$

$x_{77} := x_{42} + x_{12};$ // Aufruf vorbereiten (Makros!)

$x_{78} := \text{suc}(x_5);$

$P'_{mul};$ // Aufruf, Indices um 76 erhöht

Makros II

Beispiel: $x_{73} := (x_{42} + x_{12}) \cdot \text{suc}(x_5)$, eingebettet in P mit $I_P = 75$

```
x77 := x42 + x12;           // Aufruf vorbereiten (Makros!)  
x78 := suc(x5);  
P'mul;                       // Aufruf, Indices um 76 erhöht  
x73 := x76;                 // Ergebnis abholen
```

Makros II

Beispiel: $x_{73} := (x_{42} + x_{12}) \cdot \text{suc}(x_5)$, eingebettet in P mit $I_P = 75$

```
x77 := x42 + x12;           // Aufruf vorbereiten (Makros!)
x78 := suc(x5);
P'_{mul};                   // Aufruf, Indices um 76 erhöht
x73 := x76;                 // Ergebnis abholen
x76 := 0;    x77 := 0;      x78 := 0  // Sauber machen (Hilfsvariablen?)
```

Makros III

Bedingte Ausführung via Schleifen

Makros III

Bedingte Ausführung via Schleifen

■ **if** $x_i = 0$ **then** P **end** // wird zu:

$x_{I_P+1} := 1;$

loop x_i **do** $x_{I_P+1} := 0$ **end**;

loop x_{I_P+1} **do** P **end**;

$x_{I_P+1} := 0$

Makros III

Bedingte Ausführung via Schleifen

- **if** $x_i = 0$ **then** P **end** // wird zu:

$x_{I_P+1} := 1;$

loop x_i **do** $x_{I_P+1} := 0$ **end**;

loop x_{I_P+1} **do** P **end**;

$x_{I_P+1} := 0$

- **if** $x_i = 0$ **then** P **end** // wird (alternativ) zu:

$x_{I_P+1} := 1 - x_i;$ // Subtraktionsmakro in Übung

loop x_{I_P+1} **do** P **end**;

$x_{I_P+1} := 0$

Makros III

Bedingte Ausführung via Schleifen

- **if** $x_i = 0$ **then** P **end** // wird zu:

```
 $x_{I_P+1} := 1;$   
loop  $x_i$  do  $x_{I_P+1} := 0$  end;  
loop  $x_{I_P+1}$  do  $P$  end;  
 $x_{I_P+1} := 0$ 
```

- **if** $x_i = 0$ **then** P **end** // wird (alternativ) zu:

```
 $x_{I_P+1} := 1 - x_i;$  // Subtraktionsmakro in Übung  
loop  $x_{I_P+1}$  do  $P$  end;  
 $x_{I_P+1} := 0$ 
```

- **if** $x_i = n$ **then** P **end** // wird zu:

```
 $x_{I_P+1} := (x_i - n) + (n - x_i);$  // Subtraktionsmakro in Übung  
if  $x_{I_P+1} = 0$  then  $P$  end;  
 $x_{I_P+1} := 0$ 
```

Loop-Programme Mächtigkeit

Vergleich zur Turing-Berechenbarkeit

- Alle $f \in \mathcal{LF}$ per Konstruktion total ($\mathcal{TLF} = \mathcal{LF}$)

Vergleich zur Turing-Berechenbarkeit

- Alle $f \in \mathcal{LF}$ per Konstruktion total ($\mathcal{TLF} = \mathcal{LF}$)
- ↪ Loop-Programme terminieren zwangsweise

Vergleich zur Turing-Berechenbarkeit

- Alle $f \in \mathcal{LF}$ per Konstruktion total ($\mathcal{TLF} = \mathcal{LF}$)
- ↪ Loop-Programme terminieren zwangsweise
- Unterschied $f : \mathbb{N}^n \rightarrow \mathbb{N}$ zu $f : \mathbb{N} \rightarrow \mathbb{N}$ essentiell oder nicht?

Vergleich zur Turing-Berechenbarkeit

- Alle $f \in \mathcal{LF}$ per Konstruktion total ($\mathcal{TLF} = \mathcal{LF}$)
- ↪ Loop-Programme terminieren zwangsweise
- Unterschied $f : \mathbb{N}^n \rightarrow \mathbb{N}$ zu $f : \mathbb{N} \rightarrow \mathbb{N}$ essentiell oder nicht?
- $\mathcal{LF}$ vs $\mathcal{TLF}$:
 - $\mathcal{LF} \subseteq \mathcal{TLF}$?
 - $\mathcal{LF} \supseteq \mathcal{TLF}$?
 - $\mathcal{LF} = \mathcal{TLF}$?
 - Nichts von alledem?

While-Programme

While-Programme

Das Berechenbarkeitsmodell

Definition

- **While**-Programme bestehen aus folgenden Komponenten:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren `suc` und `pred`
 - dem Zuweisungsoperator `:=`
 - dem Kompositionsoperator `;`
 - dem Schleifenkonstrukt **while** `do end`

Definition

- **While**-Programme bestehen aus folgenden Komponenten:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren `suc` und `pred`
 - dem Zuweisungsoperator `:=`
 - dem Kompositionsoperator `;`
 - dem Schleifenkonstrukt **while** `do end`

↪ $\mathcal{WP}$ = Menge aller While-Programme

Semantik

Analog zu Loop-Programmen bis auf wenige Änderungen:

Semantik

Analog zu Loop-Programmen bis auf wenige Änderungen:

$$\blacksquare \delta :_{\subseteq} ((\mathbb{N} \rightarrow \mathbb{N}) \cup \{\perp\}) \times \mathcal{WP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

Semantik

Analog zu Loop-Programmen bis auf wenige Änderungen:

- $\delta :_{\subseteq} ((\mathbb{N} \rightarrow \mathbb{N}) \cup \{\perp\}) \times \mathcal{WP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$
- $\delta(\perp, P) = \perp$

Semantik

Analog zu Loop-Programmen bis auf wenige Änderungen:

- $\delta :_{\subseteq} ((\mathbb{N} \rightarrow \mathbb{N}) \cup \{\perp\}) \times \mathcal{WP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$
- $\delta(\perp, P) = \perp$
- Notation P^k : Programm P wird k mal kopiert ($P^0 = \varepsilon$)

Semantik

Analog zu Loop-Programmen bis auf wenige Änderungen:

- $\delta :_{\subseteq} ((\mathbb{N} \rightarrow \mathbb{N}) \cup \{\perp\}) \times \mathcal{WP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$
- $\delta(\perp, P) = \perp$
- Notation P^k : Programm P wird k mal kopiert ($P^0 = \varepsilon$)
- Menge an Iterationen bis zur Terminierung: $T_{v,P,i} := \{k \in \mathbb{N} \mid \delta(v, P^k)(i) = 0\}$

Semantik

Analog zu Loop-Programmen bis auf wenige Änderungen:

- $\delta :_{\subseteq} ((\mathbb{N} \rightarrow \mathbb{N}) \cup \{\perp\}) \times \mathcal{WP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$
- $\delta(\perp, P) = \perp$
- Notation P^k : Programm P wird k mal kopiert ($P^0 = \varepsilon$)
- Menge an Iterationen bis zur Terminierung: $T_{v,P,i} := \{k \in \mathbb{N} \mid \delta(v, P^k)(i) = 0\}$
- $\delta(v, \mathbf{while} \ x_i \ \mathbf{do} \ P \ \mathbf{end}) := \begin{cases} \perp & \text{falls } T_{v,P,i} = \emptyset, \\ \delta(v, P^{\min(T_{v,P,i})}) & \text{falls } T_{v,P,i} \neq \emptyset \end{cases}$

While-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ heißt While-berechenbar $\iff$

While-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ heißt While-berechenbar $\iff$
es gibt $P \in \mathcal{WP}$, sodass für alle $x_1, \dots, x_n \in \mathbb{N}$:

While-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ heißt While-berechenbar $\iff$
es gibt $P \in \mathcal{WP}$, sodass für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \delta((0, x_1, \dots, x_n, 0, \dots), P) = \perp)$ **ODER**

While-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ heißt While-berechenbar $\iff$
es gibt $P \in \mathcal{WP}$, sodass für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \delta((0, x_1, \dots, x_n, 0, \dots), P) = \perp)$ **ODER**
 $(f(x_1, \dots, x_n) = \delta((0, x_1, \dots, x_n, 0, \dots), P)(0) \neq \perp)$

While-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ heißt While-berechenbar $\iff$
es gibt $P \in \mathcal{WP}$, sodass für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \delta((0, x_1, \dots, x_n, 0, \dots), P) = \perp)$ **ODER**
 $(f(x_1, \dots, x_n) = \delta((0, x_1, \dots, x_n, 0, \dots), P)(0) \neq \perp)$
- $\rightsquigarrow \mathcal{WF} =$ Menge aller While-berechenbaren Funktionen
- $\rightsquigarrow \mathcal{TWF} =$ Menge aller totalen While-berechenbaren Funktionen

While-Programme Beispiel

Die konstant nicht definierte Funktion

- Programm $\text{const}_\perp$:
 $x_1 := \text{suc}(x_2); \text{while } x_1 \text{ do } x_2 := 0 \text{ end}$

Die konstant nicht definierte Funktion

- Programm $\text{const}_\perp$:
 $x_1 := \text{suc}(x_2)$; **while** x_1 **do** $x_2 := 0$ **end**
- (Programm unabhängig von Eingaben und deren Anzahl)

Die konstant nicht definierte Funktion

- Programm $\text{const}_\perp$:

$x_1 := \text{suc}(x_2); \textbf{while } x_1 \textbf{ do } x_2 := 0 \textbf{ end}$

- (Programm unabhängig von Eingaben und deren Anzahl)

↪ $\text{const}_\perp^m \in \mathcal{WF}$, wobei $\text{const}_\perp^m : \subseteq \mathbb{N}^m \rightarrow \mathbb{N}$ mit $\text{const}_\perp^m(x_1, \dots, x_m) = \perp$

While-Programme Makros

Makros I

- **loop** x_i **do** P **end** // wird zu:

$x_{I_P+1} := x_i;$

while x_{I_P+1} **do**

$P;$

$x_{I_P+1} := \text{pred}(x_{I_P+1})$

end

Makros I

- **loop** x_i **do** P **end** // wird zu:

```
 $x_{I_P+1} := x_i;$   
while  $x_{I_P+1}$  do  
     $P;$   
     $x_{I_P+1} := \text{pred}(x_{I_P+1})$   
end
```

- **while** $x_i < x_j$ **do** P **end** // wird zu:

```
 $x_{I_P+1} := x_j - x_i$   
while  $x_{I_P+1}$  do  
     $P;$   
     $x_{I_P+1} := x_j - x_i$   
end
```

Makros II

■ **while** $x_i > x_j$ **do** P **end** // wird zu:

$x_{l_P+1} := x_i - x_j$

while x_{l_P+1} **do**

$P;$

$x_{l_P+1} := x_i - x_j$

end

Makros II

- **while** $x_i > x_j$ **do** P **end** // wird zu:

```
 $x_{I_P+1} := x_i - x_j$   
while  $x_{I_P+1}$  do  
   $P$ ;  
   $x_{I_P+1} := x_i - x_j$   
end
```

- **while** $x_i = x_j$ **do** P **end** // wird zu:

```
 $x_{I_P+1} := 1 - ((x_i - x_j) + (x_j - x_i))$   
while  $x_{I_P+1}$  do  
   $P$ ;  
   $x_{I_P+1} := 1 - ((x_i - x_j) + (x_j - x_i))$   
end
```

While-Programme

Mächtigkeit

Vergleich zur Loop-Berechenbarkeit

- Enthält nicht totale Funktionen ($\mathcal{TW}\mathcal{F} \subset \mathcal{WF}$)

Vergleich zur Loop-Berechenbarkeit

- Enthält nicht totale Funktionen ($\mathcal{TW}\mathcal{F} \subset \mathcal{WF}$)

$$\rightsquigarrow \mathcal{LF} \subset \mathcal{WF}$$

Vergleich zur Loop-Berechenbarkeit

- Enthält nicht totale Funktionen ($\mathcal{TW}\mathcal{F} \subset \mathcal{WF}$)

↪ $\mathcal{LF} \subset \mathcal{WF}$

- Ist $\mathcal{TW}\mathcal{F} = \mathcal{LF}$?

Vergleich zur Loop-Berechenbarkeit

- Enthält nicht totale Funktionen ($\mathcal{TW}\mathcal{F} \subset \mathcal{WF}$)

↪ $\mathcal{LF} \subset \mathcal{WF}$

- Ist $\mathcal{TW}\mathcal{F} = \mathcal{LF}$?
- Vergleich zur Turing-Berechenbarkeit?

Goto-Programme

Goto-Programme

Das Berechenbarkeitsmodell

Definition

- **Goto**-Programme als Sequenzen $M_1 : A_1; \dots; M_n : A_n$, wobei die A_i aus folgenden Komponenten bestehen:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren suc und pred
 - dem Zuweisungsoperator $:=$

Definition

- **Goto**-Programme als Sequenzen $M_1 : A_1; \dots; M_n : A_n$, wobei die A_i aus folgenden Komponenten bestehen:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren `suc` und `pred`
 - dem Zuweisungsoperator `:=`
 - dem bedingten Sprungbefehl **if** x_i **goto** M_j für $j \neq 0$

Definition

- **Goto**-Programme als Sequenzen $M_1 : A_1; \dots; M_n : A_n$, wobei die A_i aus folgenden Komponenten bestehen:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren `suc` und `pred`
 - dem Zuweisungsoperator `:=`
 - dem bedingten Sprungbefehl **if** x_i **goto** M_j für $j \neq 0$
 - dem Stoppbefehl **halt** (Vereinbarung: Am Ende eines jeden Programms steht implizit $M_{n+1} : \mathbf{halt}$, um ein Rauslaufen zu vermeiden)

Definition

- **Goto**-Programme als Sequenzen $M_1 : A_1; \dots; M_n : A_n$, wobei die A_i aus folgenden Komponenten bestehen:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren succ und pred
 - dem Zuweisungsoperator $:=$
 - dem bedingten Sprungbefehl **if** x_i **goto** M_j für $j \neq 0$
 - dem Stoppbefehl **halt** (Vereinbarung: Am Ende eines jeden Programms steht implizit $M_{n+1} : \text{halt}$, um ein Rauslaufen zu vermeiden)
- ⇒ $\mathcal{GP}$ = Menge aller **Goto**-Programme

Semantik

Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ definiert als:

Semantik

Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ definiert als:

- Kommt $M_k : \mathbf{halt}$ in P vor, so
 $(v, k) \vdash_P (v, 0)$

Semantik

Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ definiert als:

- Kommt $M_k : \mathbf{halt}$ in P vor, so
 $(v, k) \vdash_P (v, 0)$
- Kommt $M_k : A_k$ mit $A_k \in \{x_i := 0, x_i := \text{suc}(x_j), x_i := \text{pred}(x_j)\}$ in P vor, so

Semantik

Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ definiert als:

- Kommt $M_k : \mathbf{halt}$ in P vor, so
 $(v, k) \vdash_P (v, 0)$
- Kommt $M_k : A_k$ mit $A_k \in \{x_i := 0, x_i := \text{suc}(x_j), x_i := \text{pred}(x_j)\}$ in P vor, so
 $(v, k) \vdash_P (\delta_{\text{Loop}}(v, A_k), k + 1)$

Semantik

Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ definiert als:

- Kommt $M_k : \mathbf{halt}$ in P vor, so
 $(v, k) \vdash_P (v, 0)$
- Kommt $M_k : A_k$ mit $A_k \in \{x_i := 0, x_i := \text{suc}(x_j), x_i := \text{pred}(x_j)\}$ in P vor, so
 $(v, k) \vdash_P (\delta_{\text{Loop}}(v, A_k), k + 1)$
- Kommt $M_k : \mathbf{if } x_i \mathbf{ goto } M_l$ in P vor, so

Semantik

Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ definiert als:

- Kommt $M_k : \mathbf{halt}$ in P vor, so
 $(v, k) \vdash_P (v, 0)$
- Kommt $M_k : A_k$ mit $A_k \in \{x_i := 0, x_i := \text{suc}(x_j), x_i := \text{pred}(x_j)\}$ in P vor, so
 $(v, k) \vdash_P (\delta_{\text{Loop}}(v, A_k), k + 1)$
- Kommt $M_k : \mathbf{if } x_i \mathbf{ goto } M_l$ in P vor, so

$$(v, k) \vdash_P \begin{cases} (v, l) & \text{falls } x_i \neq 0, \\ (v, k + 1) & \text{falls } x_i = 0 \end{cases}$$

Semantik

Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ definiert als:

- Kommt $M_k : \mathbf{halt}$ in P vor, so
 $(v, k) \vdash_P (v, 0)$
- Kommt $M_k : A_k$ mit $A_k \in \{x_i := 0, x_i := \text{suc}(x_j), x_i := \text{pred}(x_j)\}$ in P vor, so
 $(v, k) \vdash_P (\delta_{\text{Loop}}(v, A_k), k + 1)$
- Kommt $M_k : \mathbf{if } x_i \mathbf{ goto } M_l$ in P vor, so

$$(v, k) \vdash_P \begin{cases} (v, l) & \text{falls } x_i \neq 0, \\ (v, k + 1) & \text{falls } x_i = 0 \end{cases}$$

↪ Reflexiv-transitiver Abschluss $\vdash_P^*$

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2 \quad // \text{ Fixer Sprungbefehl!}$

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2 \quad // \text{ Fixer Sprungbefehl!}$

- $((0, 3, 4, 0, \dots), 1) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 2) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 4) \vdash_{\text{add}}$

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2 \quad // \text{ Fixer Sprungbefehl!}$

- $((0, 3, 4, 0, \dots), 1) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 2) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 4) \vdash_{\text{add}}$
 $((5, 3, 4, 0, \dots), 5) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 6) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 2) \vdash_{\text{add}}$

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2 \quad // \text{ Fixer Sprungbefehl!}$

- $((0, 3, 4, 0, \dots), 1) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 2) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 4) \vdash_{\text{add}}$
 $((5, 3, 4, 0, \dots), 5) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 6) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 2) \vdash_{\text{add}}$
 $((5, 2, 4, 0, \dots), 4) \vdash_{\text{add}} ((6, 2, 4, 0, \dots), 5) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 6) \vdash_{\text{add}}$

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2 \quad // \text{ Fixer Sprungbefehl!}$

- $((0, 3, 4, 0, \dots), 1) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 2) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 4) \vdash_{\text{add}}$
 $((5, 3, 4, 0, \dots), 5) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 6) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 2) \vdash_{\text{add}}$
 $((5, 2, 4, 0, \dots), 4) \vdash_{\text{add}} ((6, 2, 4, 0, \dots), 5) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 6) \vdash_{\text{add}}$
 $((6, 1, 4, 0, \dots), 2) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 4) \vdash_{\text{add}} ((7, 1, 4, 0, \dots), 5) \vdash_{\text{add}}$

Addition I

■ Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2 \quad // \text{ Fixer Sprungbefehl!}$

- $((0, 3, 4, 0, \dots), 1) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 2) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 4) \vdash_{\text{add}}$
 $((5, 3, 4, 0, \dots), 5) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 6) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 2) \vdash_{\text{add}}$
 $((5, 2, 4, 0, \dots), 4) \vdash_{\text{add}} ((6, 2, 4, 0, \dots), 5) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 6) \vdash_{\text{add}}$
 $((6, 1, 4, 0, \dots), 2) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 4) \vdash_{\text{add}} ((7, 1, 4, 0, \dots), 5) \vdash_{\text{add}}$
 $((7, 0, 4, 0, \dots), 6) \vdash_{\text{add}} ((7, 0, 4, 0, \dots), 2) \vdash_{\text{add}} ((7, 0, 4, 0, \dots), 3) \vdash_{\text{add}}$

Addition I

■ Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2 \quad // \text{ Fixer Sprungbefehl!}$

- $((0, 3, 4, 0, \dots), 1) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 2) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 4) \vdash_{\text{add}}$
 $((5, 3, 4, 0, \dots), 5) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 6) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 2) \vdash_{\text{add}}$
 $((5, 2, 4, 0, \dots), 4) \vdash_{\text{add}} ((6, 2, 4, 0, \dots), 5) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 6) \vdash_{\text{add}}$
 $((6, 1, 4, 0, \dots), 2) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 4) \vdash_{\text{add}} ((7, 1, 4, 0, \dots), 5) \vdash_{\text{add}}$
 $((7, 0, 4, 0, \dots), 6) \vdash_{\text{add}} ((7, 0, 4, 0, \dots), 2) \vdash_{\text{add}} ((7, 0, 4, 0, \dots), 3) \vdash_{\text{add}}$
 $((7, 0, 4, 0, \dots), 0)$

Goto-Berechenbarkeit

■ $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ Goto-berechenbar $\iff$ es gibt $P \in \mathcal{GP}$, für alle $x_1, \dots, x_n \in \mathbb{N}$:

Goto-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ Goto-berechenbar $\iff$ es gibt $P \in \mathcal{GP}$, für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \perp, \text{ es gibt kein } v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0))$

Goto-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ Goto-berechenbar $\iff$ es gibt $P \in \mathcal{GP}$, für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \perp, \text{ es gibt kein } v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0))$
ODER

Goto-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ Goto-berechenbar $\iff$ es gibt $P \in \mathcal{GP}$, für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \perp, \text{ es gibt kein } v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0))$
ODER
 $(\text{es gibt } v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0),$

Goto-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ Goto-berechenbar $\iff$ es gibt $P \in \mathcal{GP}$, für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \perp, \text{ es gibt kein } v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0))$
ODER
 $(\text{es gibt } v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0),$
 $f(x_1, \dots, x_n) = v(0) \neq \perp)$

Goto-Berechenbarkeit

- $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ Goto-berechenbar $\iff$ es gibt $P \in \mathcal{GP}$, für alle $x_1, \dots, x_n \in \mathbb{N}$:
 $(f(x_1, \dots, x_n) = \perp, \text{es gibt kein } v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0))$
ODER
 (es gibt $v : \mathbb{N} \rightarrow \mathbb{N}, ((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0),$
 $f(x_1, \dots, x_n) = v(0) \neq \perp$)

$\rightsquigarrow \mathcal{GF}$ = Menge aller Goto-berechenbaren Funktionen

$\rightsquigarrow \mathcal{TGF}$ = Menge aller totalen Goto-berechenbaren Funktionen

Addition II

- Allgemein: $((0, n, m, 0, \dots), 1) \vdash_{\text{add}}^* ((n + m, 0, m, 0, \dots), 0)$

Addition II

■ Allgemein: $((0, n, m, 0, \dots), 1) \vdash_{\text{add}}^* ((n + m, 0, m, 0, \dots), 0)$

$\rightsquigarrow \text{add} \in \mathcal{TGF}$

Goto-Programme Makros

Makros

- Bisher Abkürzungen ohne Nebenwirkungen
z. B. ändert $x_3 := (x_1 + x_5) \cdot x_7$ nicht einfach x_{12}

Makros

- Bisher Abkürzungen ohne Nebenwirkungen
z. B. ändert $x_3 := (x_1 + x_5) \cdot x_7$ nicht einfach x_{12}
- ⇒ Wegen Sprünge müssten hier eigentlich jeweilige Äquivalenzen zum Originalmodell nachgewiesen werden

Makros

- Bisher Abkürzungen ohne Nebenwirkungen
z. B. ändert $x_3 := (x_1 + x_5) \cdot x_7$ nicht einfach x_{12}
- ⇒ Wegen Sprünge müssten hier eigentlich jeweilige Äquivalenzen zum Originalmodell nachgewiesen werden
- Geben nur Skizzen an, die die Simulation grob aufzeigen

Makros

- Bisher Abkürzungen ohne Nebenwirkungen
z. B. ändert $x_3 := (x_1 + x_5) \cdot x_7$ nicht einfach x_{12}
- ↪ Wegen Sprünge müssten hier eigentlich jeweilige Äquivalenzen zum Originalmodell nachgewiesen werden
- Geben nur Skizzen an, die die Simulation grob aufzeigen
- ↪ Annahme: Variablen, die in Makro nicht auftauchen, sind neu

Makros

■ **goto** M_k // wird zu:

$x_j := 1;$

if x_j **goto** M_k

Makros

- **goto** M_k // wird zu:

$x_j := 1;$

if x_j **goto** M_k

- **if** $x_1 < x_2$ **goto** M_k // wird zu:

$x_j := x_2 - x_1;$

if x_j **goto** M_k

Makros

- **goto** M_k // wird zu:

$x_j := 1;$

if x_j **goto** M_k

- **if** $x_1 < x_2$ **goto** M_k // wird zu:

$x_j := x_2 - x_1;$

if x_j **goto** M_k

- **if** $x_1 > x_2$ **goto** M_k // wird zu:

$x_j := x_1 - x_2;$

if x_j **goto** M_k

Makros

- **goto M_k //** wird zu:

$x_j := 1;$
if x_j goto M_k

- **if $x_1 < x_2$ goto M_k //** wird zu:

$x_j := x_2 - x_1;$
if x_j goto M_k

- **if $x_1 > x_2$ goto M_k //** wird zu:

$x_j := x_1 - x_2;$
if x_j goto M_k

- **if $x_1 = x_2$ goto M_k //** wird zu:

$x_j := 1 - ((x_2 - x_1) + (x_1 - x_2));$
if x_j goto M_k

Goto-Programme

Mächtigkeit

Vergleich zur While-Berechenbarkeit I

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Vergleich zur While-Berechenbarkeit I

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:
 - Sei $f \in \mathcal{WF}$

Vergleich zur While-Berechenbarkeit I

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:
 - Sei $f \in \mathcal{WF}$
 - ↪ Daher gibt es $P_W \in \mathcal{WP}$ der Form $P_W = A_1; \dots; A_n$, das diese Funktion berechnet

Vergleich zur While-Berechenbarkeit I

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\supseteq$ ”:

■ Sei $f \in \mathcal{WF}$

↪ Daher gibt es $P_W \in \mathcal{WP}$ der Form $P_W = A_1; \dots; A_n$, das diese Funktion berechnet

■ Konstruiere zunächst $M_1 : A_1; \dots; M_n : A_n; M_{n+1} : \mathbf{halt}$

■ Ersetze jedes $M_i : \mathbf{while } x_i \mathbf{ do } P \mathbf{ end}$ durch

$M_i : \mathbf{if } x_i = 0 \mathbf{ goto } M_{i+1};$

$P;$

$M_{\text{new}_i} : \mathbf{goto } M_i;$

$M_{i+1} : \dots$

Vergleich zur While-Berechenbarkeit II

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:
 - ... Übersetze P rekursiv durch ein äquivalentes $P' \in \mathcal{GP}$, aber lasse dabei den letzten halt-Befehl weg

Vergleich zur While-Berechenbarkeit II

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:
 - ... Übersetze P rekursiv durch ein äquivalentes $P' \in \mathcal{GP}$, aber lasse dabei den letzten halt-Befehl weg
 - Baue fehlende Marker ein, shifte die Nummerierung der alten Marker entsprechend und passe auch die goto-Befehle an, um P_G zu erhalten

Vergleich zur While-Berechenbarkeit II

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:

- ... Übersetze P rekursiv durch ein äquivalentes $P' \in \mathcal{GP}$, aber lasse dabei den letzten halt-Befehl weg
- Baue fehlende Marker ein, shifte die Nummerierung der alten Marker entsprechend und passe auch die goto-Befehle an, um P_G zu erhalten
- Weise nach, dass $f_{P_G} = f_{P_W} = f$

Vergleich zur While-Berechenbarkeit III

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Vergleich zur While-Berechenbarkeit III

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”:
 - Sei $f \in \mathcal{GF}$

Vergleich zur While-Berechenbarkeit III

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”:

■ Sei $f \in \mathcal{GF}$

↪ Dann gibt es $P_G \in \mathcal{GP}$ mit $P_G = M_1 : A_1; \dots; M_n : A_n$, das f berechnet

Vergleich zur While-Berechenbarkeit III

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”:

■ Sei $f \in \mathcal{GF}$

⇒ Dann gibt es $P_G \in \mathcal{GP}$ mit $P_G = M_1 : A_1; \dots; M_n : A_n$, das f berechnet

■ Konstruktion von P_W mit Extra-Variable y , um Marker zu simulieren, sowie einer (sichtbaren) While-Schleife zur Fallunterscheidung ...

Vergleich zur While-Berechenbarkeit IV

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”: ...

```
y := 1;  
while y  $\neq$  0 do  
  if y = 1 then  
    ... //  $A_1$   
  end;  
  ...  
  if y = n then  
    ... //  $A_n$   
  end  
end
```

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”:
 - ... Übersetze $M_k : A_k$:

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”:
 - ... Übersetze $M_k : A_k$:
 - $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”:

■ ... Übersetze $M_k : A_k$:

■ $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$

■ $M_k : x_i := \text{succ}(x_j)$ durch $x_i := \text{succ}(x_j); y := k + 1$

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”:

■ ... Übersetze $M_k : A_k$:

- $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$
- $M_k : x_i := \text{succ}(x_j)$ durch $x_i := \text{succ}(x_j); y := k + 1$
- $M_k : x_i := \text{pred}(x_j)$ durch $x_i := \text{pred}(x_j); y := k + 1$

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”:

■ ... Übersetze $M_k : A_k$:

- $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$
- $M_k : x_i := \text{succ}(x_j)$ durch $x_i := \text{succ}(x_j); y := k + 1$
- $M_k : x_i := \text{pred}(x_j)$ durch $x_i := \text{pred}(x_j); y := k + 1$
- $M_k : \text{if } x_i \text{ goto } M_l$ durch $y := k + 1; \text{if } x_i \text{ then } y := l \text{ end}$

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

■ “ $\subseteq$ ”:

■ ... Übersetze $M_k : A_k$:

- $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$
- $M_k : x_i := \text{succ}(x_j)$ durch $x_i := \text{succ}(x_j); y := k + 1$
- $M_k : x_i := \text{pred}(x_j)$ durch $x_i := \text{pred}(x_j); y := k + 1$
- $M_k : \text{if } x_i \text{ goto } M_l$ durch $y := k + 1; \text{if } x_i \text{ then } y := l \text{ end}$
- $M_k : \text{halt}$ durch $y := 0$

Vergleich zur While-Berechenbarkeit V

Theorem 1.

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”:

- ... Übersetze $M_k : A_k$:

- $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$
- $M_k : x_i := \text{succ}(x_j)$ durch $x_i := \text{succ}(x_j); y := k + 1$
- $M_k : x_i := \text{pred}(x_j)$ durch $x_i := \text{pred}(x_j); y := k + 1$
- $M_k : \text{if } x_i \text{ goto } M_l$ durch $y := k + 1; \text{if } x_i \text{ then } y := l \text{ end}$
- $M_k : \text{halt}$ durch $y := 0$

- Weise nach, dass $f_{P_W} = f_{P_G} = f$

Nebenergebnis und Korollar

Theorem 2 (Satz von Kleene).

Jede While-berechenbare Funktion lässt sich – abgesehen von Loop-Simulationen – durch ein While-Programm mit nur einer While-Schleife berechnen

Nebenergebnis und Korollar

Theorem 2 (Satz von Kleene).

Jede While-berechenbare Funktion lässt sich – abgesehen von Loop-Simulationen – durch ein While-Programm mit nur einer While-Schleife berechnen

Korollar.

$$\mathcal{TGF} = \mathcal{TWF}$$

Vergleich zur Turing-Berechenbarkeit I

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Vergleich zur Turing-Berechenbarkeit I

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:

Vergleich zur Turing-Berechenbarkeit I

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:
 - Sei $f \in \mathcal{WF}$
 - ↪ Dann gibt es P_W , das dieses berechnet

Vergleich zur Turing-Berechenbarkeit I

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:

- Sei $f \in \mathcal{WF}$

- ↪ Dann gibt es P_W , das dieses berechnet

- ↪ Wandle dieses gemäß der Konstruktion von Theorem 2 zunächst in P_G und dann in P'_W um; sodass P'_W von der folgenden Form ist (A_i ohne echte While-Schleifen): ...

Vergleich zur Turing-Berechenbarkeit II

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ “ $\subseteq$ ”: ...

```
y := 1;  
while y  $\neq$  0 do  
  if y = 1 then  
    ... //  $A_1$   
  end;  
  ...  
  if y = n then  
    ... //  $A_n$   
  end  
end
```

Vergleich zur Turing-Berechenbarkeit III

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$: ...
 - Simulation mit TM M mit $I_{P'_W} + 1$ Bändern: eins für jede Variable und ein Band zum Herunterzählen für Loop-Schleifen

Vergleich zur Turing-Berechenbarkeit III

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$: ...
 - Simulation mit TM M mit $I_{P'_W} + 1$ Bändern: eins für jede Variable und ein Band zum Herunterzählen für Loop-Schleifen
 - Konstruktion von Elementarmaschinen zum Nullsetzen ($\text{Init}(x_i)$), Kopieren ($\text{Copy}(x_i, x_j)$), bilden des Vorgängers ($\text{Pred}(x_i)$) und des Nachfolgers ($\text{Suc}(x_i)$)

Vergleich zur Turing-Berechenbarkeit III

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{WF} \subseteq \mathcal{TF}$: ...

- Simulation mit TM M mit $I_{P'_W} + 1$ Bändern: eins für jede Variable und ein Band zum Herunterzählen für Loop-Schleifen
- Konstruktion von Elementarmaschinen zum Nullsetzen ($\text{Init}(x_i)$), Kopieren ($\text{Copy}(x_i, x_j)$), bilden des Vorgängers ($\text{Pred}(x_i)$) und des Nachfolgers ($\text{Suc}(x_i)$)

~> Aufbauend darauf lassen sich TMs konstruieren für die Hintereinanderausführung sowie Loop-Schleifen (mittels des Hilfsbandes)

Vergleich zur Turing-Berechenbarkeit III

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{WF} \subseteq \mathcal{TF}$: ...

- Simulation mit TM M mit $I_{P'_W} + 1$ Bändern: eins für jede Variable und ein Band zum Herunterzählen für Loop-Schleifen
- Konstruktion von Elementarmaschinen zum Nullsetzen ($\text{Init}(x_i)$), Kopieren ($\text{Copy}(x_i, x_j)$), bilden des Vorgängers ($\text{Pred}(x_i)$) und des Nachfolgers ($\text{Suc}(x_i)$)

⇒ Aufbauend darauf lassen sich TMs konstruieren für die Hintereinanderausführung sowie Loop-Schleifen (mittels des Hilfsbandes)

- D.h. für jedes A_i lässt sich eine Maschine M_i konstruieren

Vergleich zur Turing-Berechenbarkeit IV

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:
 - ... Es lässt sich für jedes n eine Maschine $\text{If}(y, n)$ mit einem Ja- und einem Nein-Ausgang konstruieren, die checkt, ob $y = n$ ist

Vergleich zur Turing-Berechenbarkeit IV

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:
 - ... Es lässt sich für jedes n eine Maschine $\text{If}(y, n)$ mit einem Ja- und einem Nein-Ausgang konstruieren, die checkt, ob $y = n$ ist
 - Es lässt sich nun M wie auf der nächsten Folie dargestellt konstruieren

Vergleich zur Turing-Berechenbarkeit IV

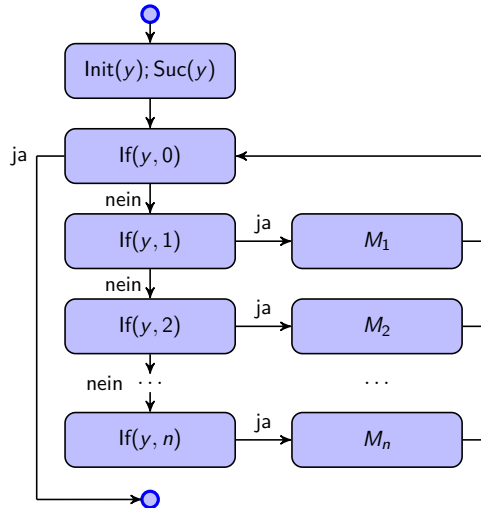
Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:
 - ... Es lässt sich für jedes n eine Maschine $\text{If}(y, n)$ mit einem Ja- und einem Nein-Ausgang konstruieren, die checkt, ob $y = n$ ist
 - Es lässt sich nun M wie auf der nächsten Folie dargestellt konstruieren
 - (Es verbleibt dann eigentlich noch zu zeigen, dass $f_M = f_{P'_W} = f_{P_W} = f$)

Vergleich zur Turing-Berechenbarkeit V



Vergleich zur Turing-Berechenbarkeit VI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Vergleich zur Turing-Berechenbarkeit VI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:
 - Sei $f \in \mathcal{TF}$

Vergleich zur Turing-Berechenbarkeit VI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$:

■ Sei $f \in \mathcal{TF}$

↪ Es gibt dann eine zugehörige unäre Funktion $f' :_{\subseteq} \{1\}^* \rightarrow \{1\}^*$ mit $f' \in \mathcal{TF}_{\{1\}}$

Vergleich zur Turing-Berechenbarkeit VI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$:

■ Sei $f \in \mathcal{TF}$

⇒ Es gibt dann eine zugehörige unäre Funktion $f' : \subseteq \{1\}^* \rightarrow \{1\}^*$ mit $f' \in \mathcal{TF}_{\{1\}}$

■ Und damit wiederum eine TM $T = (Q, \{1\}, \Gamma, \delta, q_0, B, F)$, die f' berechnet

Vergleich zur Turing-Berechenbarkeit VI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- Sei $f \in \mathcal{TF}$

- ↪ Es gibt dann eine zugehörige unäre Funktion $f' : \subseteq \{1\}^* \rightarrow \{1\}^*$ mit $f' \in \mathcal{TF}_{\{1\}}$

- Und damit wiederum eine TM $T = (Q, \{1\}, \Gamma, \delta, q_0, B, F)$, die f' berechnet

- OBdA seien $Q = \{0, \dots, n-1\}$, $\Gamma = \{0, \dots, m-1\}$ ($m \geq 2$, da $1, B \in \Gamma$) und $B=0$

Vergleich zur Turing-Berechenbarkeit VI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- Sei $f \in \mathcal{TF}$

- ↪ Es gibt dann eine zugehörige unäre Funktion $f' : \subseteq \{1\}^* \rightarrow \{1\}^*$ mit $f' \in \mathcal{TF}_{\{1\}}$

- Und damit wiederum eine TM $T = (Q, \{1\}, \Gamma, \delta, q_0, B, F)$, die f' berechnet

- OBdA seien $Q = \{0, \dots, n-1\}$, $\Gamma = \{0, \dots, m-1\}$ ($m \geq 2$, da $1, B \in \Gamma$) und $B=0$

- Grundidee für Konstruktion von P_G : Codiere Konfigurationen und simuliere die Konfigurationsübergänge

Vergleich zur Turing-Berechenbarkeit VII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:
 - ... Konfiguration (α, q, β) bei $\alpha = a_{\ell-1} \dots a_0$ und $\beta = b_0 \dots b_{k-1}$ durch

Vergleich zur Turing-Berechenbarkeit VII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$:

■ ... Konfiguration (α, q, β) bei $\alpha = a_{\ell-1} \dots a_0$ und $\beta = b_0 \dots b_{k-1}$ durch

■ $x_q := q$

■ $x_\alpha := \sum_{i=0}^{\ell-1} a_i \cdot m^i$

■ $x_\beta := \sum_{i=0}^{k-1} b_i \cdot m^i$

Vergleich zur Turing-Berechenbarkeit VII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- ... Konfiguration (α, q, β) bei $\alpha = a_{\ell-1} \dots a_0$ und $\beta = b_0 \dots b_{k-1}$ durch

- $x_q := q$

- $x_\alpha := \sum_{i=0}^{\ell-1} a_i \cdot m^i$

- $x_\beta := \sum_{i=0}^{k-1} b_i \cdot m^i$

↪ x_α und x_β m -adische Darstellungen (wegen $m \geq 2$ eindeutig)

Vergleich zur Turing-Berechenbarkeit VIII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:
 - ... Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$

Vergleich zur Turing-Berechenbarkeit VIII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- ... Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$

- Fall $\delta(i, j) = (i', j', L)$:

- $x_\beta := x_\beta \text{ div } m;$ /* Zeichen über LS-Kopf
 - $x_\beta := j' + m \cdot x_\beta;$ überschreiben */

Vergleich zur Turing-Berechenbarkeit VIII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- ... Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$

- Fall $\delta(i, j) = (i', j', L)$:

$x_\beta := x_\beta \text{ div } m;$ /* Zeichen über LS-Kopf

$x_\beta := j' + m \cdot x_\beta;$ überschreiben */

$x_\beta := x_\alpha \text{ mod } m + m \cdot x_\beta;$ /* LS-Kopf

$x_\alpha := x_\alpha \text{ div } m;$ schieben */

Vergleich zur Turing-Berechenbarkeit VIII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$:

■ ... Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$

■ Fall $\delta(i, j) = (i', j', L)$:

$x_\beta := x_\beta \text{ div } m;$ /* Zeichen über LS-Kopf

$x_\beta := j' + m \cdot x_\beta;$ überschreiben */

$x_\beta := x_\alpha \text{ mod } m + m \cdot x_\beta;$ /* LS-Kopf

$x_\alpha := x_\alpha \text{ div } m;$ schieben */

$x_q := i'$ // Folgezustand

Vergleich zur Turing-Berechenbarkeit IX

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$

- Fall $\delta(i, j) = (i', j', R)$:

$$x_\beta := x_\beta \text{ div } m;$$

$$x_\beta := j' + m \cdot x_\beta;$$

/* Zeichen über LS-Kopf
überschreiben */

Vergleich zur Turing-Berechenbarkeit IX

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:

- Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$

- Fall $\delta(i, j) = (i', j', R)$:

$x_\beta := x_\beta \text{ div } m;$ /* Zeichen über LS-Kopf

$x_\beta := j' + m \cdot x_\beta;$ überschreiben */

$x_\alpha := x_\beta \text{ mod } m + m \cdot x_\alpha;$ /* LS-Kopf

$x_\beta := x_\beta \text{ div } m;$ schieben */

Vergleich zur Turing-Berechenbarkeit IX

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$:

■ Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$

■ Fall $\delta(i, j) = (i', j', R)$:

$x_\beta := x_\beta \text{ div } m;$ /* Zeichen über LS-Kopf

$x_\beta := j' + m \cdot x_\beta;$ überschreiben */

$x_\alpha := x_\beta \text{ mod } m + m \cdot x_\alpha;$ /* LS-Kopf

$x_\beta := x_\beta \text{ div } m;$ schieben */

$x_q := i'$ // Folgezustand

Vergleich zur Turing-Berechenbarkeit X

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - Konstruiere Hauptteil von P_G (für $\delta(i, j) \neq \perp$; simuliert Konfigurationsübergänge):
 $M_2 : x_a := x_\beta \bmod m$;

Vergleich zur Turing-Berechenbarkeit X

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$: ...

■ Konstruiere Hauptteil von P_G (für $\delta(i, j) \neq \perp$; simuliert Konfigurationsübergänge):

$M_2 : x_a := x_\beta \text{ mod } m;$

if $(x_q = 1 \wedge x_a = 1)$ **goto** $M_{11};$

if $(x_q = 1 \wedge x_a = 2)$ **goto** $M_{12};$

...

if $(x_q = 2 \wedge x_a = 1)$ **goto** $M_{21};$

if $(x_q = 2 \wedge x_a = 3)$ **goto** $M_{23};$ // Hier: $\delta(2, 2) = \perp$

...

if $(x_q \in F \wedge x_\alpha = 0)$ **goto** M_3 **else goto** $M_2;$ // $\alpha = B^\ell$

Vergleich zur Turing-Berechenbarkeit XI

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

■ $\mathcal{TF} \subseteq \mathcal{GF}$: ...

■ Konstruiere Hauptteil von P_G (für $\delta(i, j) \neq \perp$; simuliert Konfigurationsübergänge):

...

$M_{11} : P_{G_{11}}; \textbf{goto } M_2;$

$M_{12} : P_{G_{12}}; \textbf{goto } M_2;$

...

$M_{21} : P_{G_{21}}; \textbf{goto } M_2;$

$M_{23} : P_{G_{23}}; \textbf{goto } M_2;$

...

Vergleich zur Turing-Berechenbarkeit XII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...

Vergleich zur Turing-Berechenbarkeit XII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - P_G muss zunächst Variablen initialisieren:
 $M_1 : x_\alpha := 0;$
 $x_q := q_0;$
 $x_\beta := \sum_{i=0}^{|Eingabe|-1} Eingabe_i \cdot m^i$

Vergleich zur Turing-Berechenbarkeit XII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - P_G muss zunächst Variablen initialisieren:
 $M_1 : x_\alpha := 0;$
 $x_q := q_0;$
 $x_\beta := \sum_{i=0}^{|Eingabe|-1} Eingabe_i \cdot m^i$
 - Dann obiger Hauptteil beginnend mit Marker M_2

Vergleich zur Turing-Berechenbarkeit XII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - P_G muss zunächst Variablen initialisieren:
 $M_1 : x_\alpha := 0;$
 $x_q := q_0;$
 $x_\beta := \sum_{i=0}^{|Eingabe|-1} Eingabe_i \cdot m^i$
 - Dann obiger Hauptteil beginnend mit Marker M_2
 - Schließlich fehlt noch Marker M_3 (Terminierung): Decodierung von x_β und Testen, ob alle b_i – wie gefordert – 1en sind, bis nur noch Blanks (0) kommen

Vergleich zur Turing-Berechenbarkeit XII

Theorem 3.

$$(\mathcal{GF} = \mathcal{WF}) = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - P_G muss zunächst Variablen initialisieren:
 $M_1 : x_\alpha := 0;$
 $x_q := q_0;$
 $x_\beta := \sum_{i=0}^{|Eingabe|-1} Eingabe_i \cdot m^i$
 - Dann obiger Hauptteil beginnend mit Marker M_2
 - Schließlich fehlt noch Marker M_3 (Terminierung): Decodierung von x_β und Testen, ob alle b_i – wie gefordert – 1en sind, bis nur noch Blanks (0) kommen
- ⇒ Eigentlich fehlt noch der Nachweis der Korrektheit der Konstruktion

Korollar für totale Berechenbarkeitsmodelle

Korollar.

$$(\mathcal{TGF} = \mathcal{TWF}) = \mathcal{TTF}$$

Zusammenfassung

- TMs (Symbole schieben) vs. Loop/While/Goto (simple Arithmetik + Kontrollstr.)

Zusammenfassung

- TMs (Symbole schieben) vs. Loop/While/Goto (simple Arithmetik + Kontrollstr.)
- Findings: $\mathcal{LF} \subseteq (\mathcal{TGF} = \mathcal{TWF} = \mathcal{TTF}) \subset (\mathcal{GF} = \mathcal{WF} = \mathcal{TF})$

Zusammenfassung

- TMs (Symbole schieben) vs. Loop/While/Goto (simple Arithmetik + Kontrollstr.)
- Findings: $\mathcal{LF} \subseteq (\mathcal{TGF} = \mathcal{TWF} = \mathcal{TTF}) \subset (\mathcal{GF} = \mathcal{WF} = \mathcal{TF})$
- Gilt auch $(\mathcal{TGF} = \mathcal{TWF} = \mathcal{TTF}) \subseteq \mathcal{LF}$?

Zusammenfassung

- TMs (Symbole schieben) vs. Loop/While/Goto (simple Arithmetik + Kontrollstr.)
 - Findings: $\mathcal{LF} \subseteq (\mathcal{TGF} = \mathcal{TWf} = \mathcal{TTf}) \subset (\mathcal{GF} = \mathcal{WF} = \mathcal{TF})$
 - Gilt auch $(\mathcal{TGF} = \mathcal{TWf} = \mathcal{TTf}) \subseteq \mathcal{LF}$?
- ↪ Treffen Berechenbarkeitsmodellen exakt die (total) berechenbaren Funktionen?